ATC Simulation Framework

User Manual

A Python / Tkinter framework for studies under varying cognitive load and task complexity

---

Developed by Ricardo Vieira

Table of Contents

1. Introduction

This framework was designed to parametrize and manipulate **cognitive load** and **task complexity** in a controlled way, each along three levels (Low/ Medium/ High), so that their effects can be studied independently or in combination. All parameters defining these levels can be edited, extended, or entirely replaced to support future manipulations or additional independent variables.

These manipulations are built into an Air Traffic Control (ATC) task, in which a participant assigns arriving flights to one of three runways (A, B, C) while acknowledging system messages that appear in a console. Alongside the task itself, the framework also handles data collection: webcam-based eye/face tracking via OpenFace, and EEG recording via a BITalino device, both started and stopped automatically for each trial, in addition to microphone audio and the event log described in Section 9.

For the current study, each participant completes a full **3 x 3 within-subjects design**: 3 levels of cognitive load crossed with 3 levels of task complexity, for 9 trial conditions of 2 minutes each. Condition order is counterbalanced across participants using precomputed Latin squares.

1.1 Who this manual is for

- Researchers who want to run the experiment with participants.
- Researchers who want to extend, adapt, or reuse the framework for a different study.
  ***For instance**, a researcher interested in studying how attention to safety alerts varies under different levels of cognitive load and task complexity, could reuse the framework, replacing only the stimulus and the three parameters defined in levels.*

2. System Requirements

This framework was developed and tested on **Windows**. The session controller uses the **pywin32** package to manage the OpenFace preview window, so it will not run as-is on macOS or Linux without modification.

2.1 Software

- Python 3.9 or later.
- Python packages: pyaudio, pywin32, bitalino (install via **pip install -r requirements.txt**).
- OpenFace 2.2.0 (eye/face tracking) -- required only if eye-tracking recording is desired.

2.2 Hardware (optional, depending on what you record)

- A webcam, for OpenFace eye/face tracking
- A BITalino device, for EEG recording (disabled by default)
- A working microphone, for audio recording (enabled by default)

  Note: EEG recording is **disabled by default** an audio recording is **enabled by default**.

3. Installation

1. Clone or download the project repository.

```
git clone <repo-url>
cd <repo-folder>
```

2. Install the required Python packages.

```
pip install -r requirements.txt
```

3. Install OpenFace 2.2.0 separately (it is not a Python package) if you intend to record eye/face tracking data -- see Section 4 for where to point the framework to it.

4. Configuration Before First Use

This framework was built for a specific lab setup, so a few values are hardcoded and must be updated before running it on a different machine. All of these are located in engine/experimentalSession.py unless noted otherwise.

What to change	Where	Details
OpenFace executable path	start_openface_recording()	Absolute path to FeatureExtraction.exe.
BITalino MAC address	start_eeg_recording()	Only relevant if use_eeg is set to True.
Microphone device index	start_audio_recording()	Defaults to index 0.

5. Project Structure

Folder / file	Contents
main.py	Entry point. Creates the Tkinter root window.
core/	Flight and Runway -- the basic simulation entities.
engine/	SimulationEngine, EventScheduler, SystemMessage(and Manager), and ExperimentalSession.
levels/	CognitiveLoadProfile, TaskComplexityProfile
ui/	ATCApp, StartMenu -- the Tkinter interface.

6. Running a Session

```
python main.py
```

6.1 Start menu

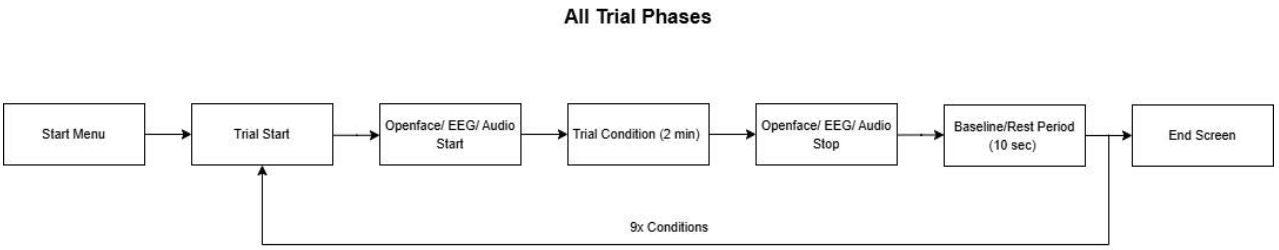
On launch, the experimenter selects a **Participant ID** (1-30) using the spinbox, then chooses one of two presented buttons.The Participant ID determines which precomputed order this participant follows for the 9 trial conditions, assigned via Latin square.

- **START** -- runs the real, data-recorded 9-condition session for that participant ID.
- **PRACTICE** -- runs a single practice condition (fixed at the lowest cognitive-load level and the highest complexity level, see Section 7.5) that loops indefinitely and does not save any data.

6.2 During a trial

Each 2-minute trial condition begins automatically once selected. All configured recordings (OpenFace, EEG if enabled, audio) start just before the trial timer, and stop automatically at the end of the condition.

A 10-second rest/baseline screen is shown between conditions. The diagram below shows all the different phases, from the start menu to the end screen.



6.3 Ending a session early

Pressing **Esc** at any point stops all recordings and returns to the start menu.

Note: If a trial is interrupted with Esc, its partial data is **not** saved to CSV. Treat any aborted trial as data-incomplete for that condition.

7. Understanding the Task

7.1 Flights and runways

Flights appear in the **Flight Queue** on the right side of the screen, each showing a callsign, an ETA (time before it must be handled), and -- depending on the trial's complexity level -- a required-runway constraint. To handle a flight, the participant selects it, selects an available runway (A, B, or C) on the left, and presses **AUTHORIZE**.

A flight that isn't assigned before its ETA reaches zero is considered **expired** (a missed/timeout error). A runway stays occupied for a fixed duration after a flight is assigned to it before it becomes available again.

7.2 System messages

System messages appear in the console below the flight queue and must be **acknowledged** (by ticking the checkbox next to them) before their own timeout elapses. Their frequency and urgency scale with the cognitive-load level of the current trial.

7.3 Priority and delayed flights

In MEDIUM and HIGH complexity conditions, waiting flights can be dynamically modified partway through the trial: promoted to **priority** (reduced ETA, shown in light yellow) or **delayed** (increased ETA, shown in grey), introducing additional unpredictability.

A priority flight, shown in light yellow with a reduced ETA.

A delayed flight, shown in grey with an increased ETA.

7.4 Runway constraints

In HIGH complexity conditions, some flights carry a **required runway** -- attempting to authorize them onto a different runway triggers a constraint-violation error and the assignment is rejected.

A constrained flight, requiring assignment to Runway A

7.5 Difficulty levels at a glance

The table below summarizes the cognitive-load parameters at each level:

Level	Event interval	Message frequency	ETA range	Dual task
LOW	9.0s	30%	22-30s	No
MEDIUM	6.6s	50%	18-26s	No
HIGH	4.25s	70%	14-22s	Yes*

The table below summarizes the task-complexity parameters at each level:

Level	Max flights	Priorities	Constraints
LOW	5	No	No
MEDIUM	6	Yes	No
HIGH	8	Yes	Yes

8. Data Output

8.1 What is collected

Before describing the file structure, it's useful to understand what each measure captures and why it's collected:

events_data - records every decision the participant makes (flight assignments, timeouts, message acknowledgements) with millisecond-level timestamps, allowing reaction times, error rates, and error types to be computed per condition. This is the primary outcome measure for the study.

eye_data - captures gaze direction, fixations, saccades, blinks, and other facial action units via OpenFace, providing a detailed record of visual attention and facial activity throughout the trial.

eeg_data - captures raw brain activity via a BITalino device, intended to support analysis of cognitive load and attentional state at a physiological level.

audio_data - records the participant's spoken acknowledgements or think-aloud comments during the trial, for later qualitative review or transcription if needed.

Each trial condition writes its own set of files, organized into folders created in the working directory. File names follow the pattern P<participant_id>_[PRACTICE_]<type><condition_index>.

Folder	Format	Contents
events_data/	CSV	Timestamped event log for the trial (see Section 9.1).
eye_data/	OpenFace output	Eye/face tracking data per condition.
eeg_data/	CSV	Raw EEG samples, start timestamp, sampling rate (if enabled).
audio_data/	WAV	Mono, 16-bit, 16kHz audio recording of the trial.

8.2 Event log codes (events_data/*.csv)

Each row is a (timestamp, event) pair. Common event codes include:

Event code	Meaning
TRIAL_START / TRIAL_END	Marks the start/end of the 2-minute condition.
FLIGHT_DT	Decision time (in seconds) for a successfully authorized flight.
FLIGHT_EXPIRED	A flight timed out before being assigned.
MESSAGE_RT_<seconds>	Reaction time for an acknowledged system message.
MESSAGE_EXPIRED	A system message timed out unacknowledged.